

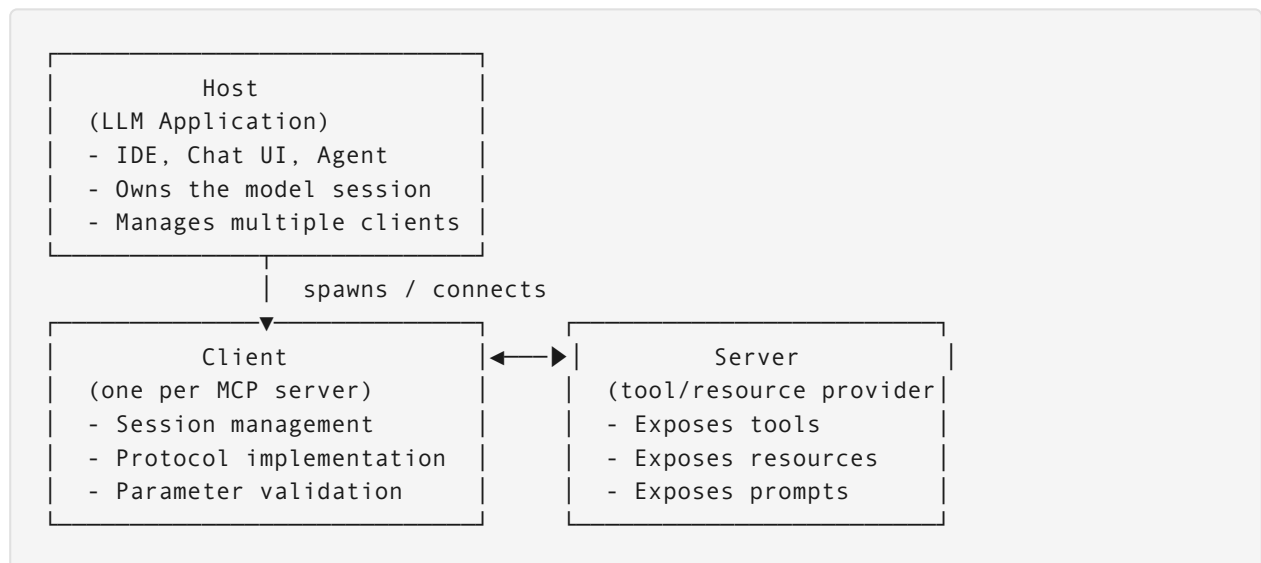
Section 11.5 of 10: Model Context Protocol (MCP)

Executive Overview

MCP standardizes AI-tool integration. Rather than every AI application implementing bespoke connectors to every data source and service, MCP defines a universal protocol that reduces $N \times M$ integration complexity to $N + M$. Understanding hosts, clients, servers, transport mechanisms, and security boundaries is essential for modern AI agent development.

11.5.1 MCP Architectural Roles

Three-Party Architecture



Why MCP Reduces Integration Complexity

Before MCP:

```
IDE  — custom connector —> GitHub
IDE  — custom connector —> Jira
IDE  — custom connector —> Postgres
...
N applications × M tools = N×M connectors to build and maintain
```

With MCP:

```

IDE (host) → MCP Client → GitHub MCP Server
                        → Jira MCP Server
                        → Postgres MCP Server

```

N hosts + M servers = N+M implementations
 Each server written once; any MCP host can use it

Role Responsibilities

Role	Owns	Responsibilities
Host	LLM session, user interface	Orchestrate clients, decide what context to expose
Client	Server connection	Protocol handshake, session state, message framing
Server	Tools and data	Implement tools, manage auth, enforce access control

11.5.2 Transport Layer Mechanics

Stdio Transport (Local)

```

Host process
├── spawns Server as child process
│   ├── stdin → Server reads JSON-RPC messages
│   └── stdout ← Server writes JSON-RPC responses
└── Communicates synchronously via pipes

```

Characteristics: - Lowest latency (no network stack) - No external network exposure (security win) - Limited to single machine; no horizontal scaling - Ideal for: IDE plugins, desktop tools, local development

HTTP/SSE Transport (Network)

```

Client                                Server
├── POST /mcp (JSON-RPC) → (process request)
└── SSE stream (event: data) ← data: {"result": ...}

```

	event: progress	
	event: complete	

Session Management:

```
GET /.well-known/mcp-configuration
→ Returns: supported methods, resource templates, auth requirements
```

Session lifecycle:

1. Client → initialize (protocol version, capabilities)
2. Server → initialized (confirm capabilities)
3. Client → tools/list, resources/list
4. Normal operation
5. Client → shutdown

Transport Comparison

Transport	Latency	Scale	Security	When to Use
Stdio	<1ms	Single machine	Process isolation	Local tools, desktop agents
HTTP + SSE	10–50ms	Multi-node	Auth + TLS required	Cloud services, shared tools

11.5.3 MCP Primitives

Tools (Model-Controlled Actions)

The model decides when and how to call tools, based on schema descriptions.

```
{
  "name": "search_files",
  "description": "Search for files matching a glob pattern in a directory",
  "inputSchema": {
    "type": "object",
    "properties": {
      "pattern": {
        "type": "string",
        "description": "Glob pattern, e.g. '**/*.py'"
      },
      "path": {
        "type": "string",
        "description": "Root directory to search from"
      }
    }
  }
}
```

```

    },
    "required": ["pattern"]
  }
}

```

Invocation flow: 1. Model selects tool based on description + schema 2. Host validates parameters against schema (rejects before server call) 3. Host calls server-side function 4. Result injected into model context as tool_result 5. Model generates next action or final response

Resources (Application-Controlled Context)

The host (not the model) decides what resources to expose. Resources are read-only data contexts.

Resource types:

```

file:///path/to/document.txt    → File contents
db://users/user_123            → Database record
api://weather/current?city=NYC → API response

```

Security model:

- Host controls access; model requests, host decides
- Resources aren't automatically in context (explicit injection)
- Provides a clear authorization boundary

Prompts (User-Controlled Workflow Templates)

Pre-built prompts that users invoke to initiate structured workflows.

```

{
  "name": "code-review",
  "description": "Review code changes for quality and security",
  "arguments": [
    {"name": "language", "required": true, "description": "Programming language"},
    {"name": "style_guide", "required": false, "description": "Style guide URL or name"}
  ]
}

```

Primitive Selection Guide

Primitive	Controlled By	Direction	Use When
Tools	Model (LLM)	Model → world	Agent needs to take actions
Resources	Application (Host)	World → model	

Primitive	Controlled By	Direction	Use When
			Providing authoritative data context
Prompts	User	User → model	Structured task initiation

11.5.4 MCP Security Model

Defense-in-Depth for MCP Servers

Layer 1: Schema-first input validation

```
# Never accept raw SQL from the model
# Tool schema restricts what the model can express

{
  "name": "query_users",
  "inputSchema": {
    "properties": {
      "status": {"enum": ["active", "inactive"]}, # No arbitrary strings
      "min_age": {"type": "integer", "minimum": 0, "maximum": 120}
    }
  }
}
# Model cannot inject SQL – it can only choose from enum values
```

Layer 2: Parameterized queries

```
def execute_query(status: str, min_age: int):
    # Still use parameterized query even with schema validation
    # Defense-in-depth principle
    return db.execute(
        "SELECT * FROM users WHERE status = ? AND age >= ?",
        [status, min_age] # Never string format into SQL
    )
```

Layer 3: Row-level security

Database connection uses session-scoped credentials
RLS policies enforce: tenant isolation, data classification, read-only for model sessions

Layer 4: Rate limiting + audit logging

```
@rate_limit(requests_per_minute=100, per="session_id")
def handle_tool_call(tool_name: str, params: dict):
    audit_log.write({
        "timestamp": now(),
        "session": session_id,
        "tool": tool_name,
        "params": params # Logged before execution
    })
    return execute_tool(tool_name, params)
```

Interview Problems

Problem 1: Design a Secure MCP Database Server

Scenario: Expose a customer database via MCP. Prevent SQL injection, enforce tenant isolation, and limit read access.

Solution:

Security architecture:

1. Schema-constrained tools only:
 - query_customers(status, min_age, max_age) → No raw SQL
 - get_customer(customer_id) → Single record lookup
 - list_orders(customer_id, date_range) → Parameterized join
2. Tenant isolation:
 - MCP session carries tenant_id (from auth token)
 - All queries auto-append: WHERE tenant_id = :session_tenant
 - No tool allows tenant_id as a parameter (always from session)
3. Read-only enforcement:
 - Database user has SELECT privileges only
 - Any INSERT/UPDATE/DELETE → authorization error
 - Schema access limited to specific tables
4. Capability scoping per session:
 - Support role: query_customers, get_customer
 - Analytics role: list_orders + aggregation tools
 - Admin role: all tools (separate audit trail)

5. Audit trail:
 - Every tool call logged to tamper-evident store
 - Alerts on: >50 queries/minute, access to >1000 records

Problem 2: MCP Server Discovery and Versioning

Scenario: Your org runs 20 MCP servers. A new version of the filesystem server changes a tool's parameter schema. How do you manage compatibility?

Solution:

Version management strategy:

1. Semantic versioning in .well-known/mcp-configuration:
`{"protocol_version": "2025-03-26", "server_version": "2.1.0"}`
2. Backward-compatible changes (minor version):
 - Adding optional parameters: safe
 - Adding new tools: safe
 - Expanding enum values: safe
3. Breaking changes (major version):
 - Renaming required parameters → needs migration
 - Removing tools → deprecation window required

4. Migration pattern for parameter rename:
Tool v1: `read_file(path)`
Tool v2: `read_file(file_path)` ← renamed

Transition period: Accept both parameter names
`def read_file(params):`
 `path = params.get("file_path") or params.get("path")`

Sunset: After 60 days, remove legacy parameter support

5. Discovery:
Internal service registry lists all MCP servers + versions
Clients negotiate version during initialize handshake
Incompatible version → return error with supported versions list

Problem 3: MCP Tool Selection Accuracy Degradation

Scenario: An agent with 30+ MCP tools has tool selection accuracy drop from 95% to 65%. How do you diagnose and fix it?

Solution:

Diagnosis:

- Log model tool selections vs. correct tool (human labeled)
- Cluster errors: wrong category? wrong variant? hallucinated tool?

Common root causes:

1. Tool descriptions too similar (e.g., "search_files" vs "find_files")
2. Too many tools exceed model's effective context window for tool schemas
3. Tool names not descriptive enough

Fixes:

Fix 1 – Description clarity:

Before: "search_files: Search for files"

After: "search_files: Find files by name/glob pattern. Use for locating files when you know part of the filename. NOT for searching file contents – use search_content for that."

Fix 2 – Tool namespace partitioning:

Rather than 30 flat tools, group into namespaces:

filesystem/* → 5 tools
database/* → 8 tools
email/* → 4 tools

Route query to relevant namespace first (meta-tool or classification), then present only that namespace's tools to the model (~5-8 tools)

Result: Tool selection accuracy returns to 90-95%

Fix 3 – Retrieval-based tool injection:

Embed all tool descriptions

At query time: retrieve top-5 most relevant tools

Only inject retrieved tools into context

Model never sees irrelevant tools